CivicResolve
Project Completion Report

Repository-wide implementation and readiness assessment

Prepared 25 October 2026

OVERALL ASSESSMENT

Core complaint intake and case-management foundations are implemented, including role-aware access, AI-assisted categorization, ticket tracking, and OTP/password flows. The project is not yet production-ready: several advertised capabilities are UI-only or absent, privileged role provisioning needs remediation, and the current full test run is blocked by application-context database configuration.

Project snapshot

Area	Observed
Application	Java 21 / Spring Boot 4.1.1; MVC + REST; Thymeleaf views
Persistence	Spring Data JPA / Hibernate with MySQL connector
Repository inventory	76 Java files, 36 Thymeleaf templates, 7 CSS files, 2 JavaScript files
Automated tests	10 test classes; latest run: 86 tests, 77 passed, 9 errored
Readiness	MVP foundation; deploy only after listed security, integration, and feature gaps are addressed

Review basis: source inventory, application configuration, README and Maven dependencies, controllers/services/entities, selected UI templates and scripts, existing test reports, and a fresh `mvnw test` run. This is an implementation/readiness review, not a penetration test or a live deployment verification.

1. Architecture and implemented workflow

The application follows a conventional layered Spring structure. Thymeleaf MVC routes render citizen, officer, administrator, landing, and complaint-reporting pages. REST controllers expose complaint, authentication, and OTP operations. Service classes own account and complaint workflows; repositories and JPA entities persist users, complaints, escalation records, OTP challenges, and password-reset tokens.

Layer	Observed implementation
Presentation	Thymeleaf templates, shared fragments, role-specific pages, CSS/JS assets; report wizard uses geolocation and browser speech features.
Application/API	MainController, AuthController, ComplaintController, OtpController; request/response DTOs and centralized exception handling.
Business logic	ComplaintServiceImpl for create/classify/assign/status/tracking; UserServiceImpl for registration, phone changes, password reset, officer provisioning.
Integrations	Gemini classification via configurable REST client; SMTP email; Twilio/mock OTP sender; external map/geocoding resources.
Security	Spring Security form login, server sessions, role-based route rules, BCrypt password encoding, remember-me support.

Workflow supported in code

- A citizen can register/sign in, submit a complaint, receive a generated or supplied ticket identifier, and retrieve a limited public tracking response.
- Complaint intake invokes Gemini categorization when configured and applies a local fallback based on the selected civic sector when AI is unavailable.
- Administrators can assign complaints; officers/admins can update status and notes; authorization checks scope individual complaint access by role/ownership/assignment.
- Account flows include password hashing, email registration OTP support, forgot/reset password, and citizen phone verification. OTP enforcement is configurable and disabled by default in the checked-in properties.

Capability status

Capability	Assessment	Evidence / qualification
Complaint intake and tracking	Implemented	REST create, role-scoped list/detail, public tracking, status and assignment actions.
AI classification	Implemented with fallback	Gemini integration, timeouts/retries, sector registry and fallback behavior. Requires external API key for AI operation.
Account and OTP workflows	Implemented; configuration-dependent	Registration/login/reset plus email/SMS OTP components. Delivery depends on SMTP/Twilio and settings.
Map/location capture	Partial	Browser geolocation/reverse-geocoding support; location is ultimately persisted as complaint text and optional coordinates.
Photo evidence	Not complete	The report screen explicitly labels the image as preview-only; submission payload is JSON and does not attach the file.
Admin analytics / department tools	Not complete	Admin dashboard reports no metrics; department page has no configured records.
Escalation and notifications	Partial / UI stubs	Escalation entity/repository and pages exist, but no end-to-end escalation API/service or persisted notification workflow was found.
Community voting	Not implemented	No voting entity, endpoint, or service identified in the code inventory.

2. Completion gaps and documentation accuracy

The README describes useful product goals, but some are aspirational rather than complete. The following discrepancies should be resolved either by implementing the behavior or by updating product documentation and acceptance criteria.

README / surface	Finding	Suggested completion action
Community voting	No data model or working API/UX behavior identified.	Implement voting with authorization, deduplication, and tests; otherwise remove the feature claim.
Photo/media uploads	Preview-only file selection; complaint API accepts JSON without an upload field/endpoint.	Add validated persistent upload and secure retrieval, or clearly label as a future feature.
Analytics and reporting	Admin dashboard shows no metrics; no reporting pipeline observed.	Define operational metrics and build data-backed dashboard/reporting.
Automated assignment/escalation	Assignment endpoint is admin-driven; no automatic policy/trigger workflow found. Escalation is represented in persistence but not end-to-end.	Define rules, lifecycle, ownership, alerts, and tested scheduled/event-driven processing.
Notifications	Citizen/officer views show empty-state content and actions are not backed by a notification store.	Implement event generation, persistence, read state, and delivery preferences.
JWT and API docs	README names JWT; configuration uses session-based form login. README points to Swagger UI, but no OpenAPI/Swagger dependency is present in `pom.xml`.	Document the actual session model or implement JWT intentionally; add API docs only if supported.
License	README states MIT, but no LICENSE file is tracked.	Add the intended license file or correct the statement.

Production and operational risks

- **Privileged role provisioning:** the account creation workflow accepts role selection and service logic maps privileged role values to administrator/officer accounts. Public self-registration must assign a citizen role only; privileged accounts should be provisioned through an explicitly authorized process.
- **Security defaults:** API CSRF exclusions are broad while authentication is session/cookie-based; the remember-me configuration uses an in-repository fixed key. Review session/CSRF design and externalize/rotate signing keys before deployment.
- **Runtime configuration:** checked-in defaults use Hibernate schema auto-update, SQL output, and verbose Spring Security logging. Establish environment-specific production profiles, controlled schema migrations, and safe logging levels.
- **External services:** database, SMTP, Twilio, Gemini, browser geolocation, speech recognition, and external frontend assets need explicit deployment configuration and availability/fallback checks.

These items are readiness findings from static review; severity and exploitability have not been independently validated through penetration testing.

3. Validation evidence and recommended plan

Fresh validation executed with the repository Maven wrapper using `mvnw test`. Maven compiled and ran the test suites, but the overall test goal exited unsuccessfully.

Result	Count / note
Total discovered tests	86 across 10 test classes
Passed	77; no assertion failures recorded
Errored	9; one application context test plus eight security/registration integration tests
Observed common cause	Spring context could not initialize the JPA entity manager because Hibernate could not determine the SQL dialect from JDBC metadata. The default test setup needs a reachable/configured database or an isolated test database profile.

Prioritized completion plan

Priority	Work item	Exit evidence
P0 - before exposure	Constrain public registration to citizen role; move privileged account creation to authorized provisioning. Review session, CSRF, remember-me key, and production security logging/config.	Negative tests prove anonymous registration cannot create privileged roles; security configuration review passes.
P1 - reliable delivery	Add a reproducible test profile using a disposable database (for example Testcontainers or H2 if compatible); make all tests pass in CI.	Clean `mvnw test` succeeds without developer-local database state.
P1 - feature completion	Implement or remove claims for uploads, voting, escalations, notifications, analytics, and departments; define acceptance criteria for each.	End-to-end tests and UI behavior confirm persisted data and permissions.
P1 - operations	Add environment-specific configuration, schema migration strategy, health checks for required integrations, and deployment/runbook guidance.	Staging deployment checklist verified; no local-only secrets or debug settings.
P2 - docs and maintainability	Align README with session-based auth and real feature status; add license and actual API documentation if intended.	Documentation matches runtime behavior; license and API route descriptions are present.

Completion decision

CivicResolve is a credible MVP foundation, not a completed production product. The core complaint workflow is substantially present, but feature parity, role provisioning, reproducible integration tests, and production configuration remain necessary gates. Treat the priorities above as release blockers until verified.

Reviewed areas included `README.md`, `pom.xml`, `application.properties`, Java controllers/services/entities/configuration, all top-level source inventories, selected templates and JavaScript, and Surefire reports. The report intentionally excludes credentials and does not claim live service or browser verification.